Análisis Avanzado de Clientes y Ventas

CURSO DE
PROGRAMACIÓN EN PYTHON
</>

MUSK

El proyecto consiste en desarrollar un script en Python que procese datos de **clientes** (JSON) y **ventas** (CSV). Se deberán realizar **10 cálculos diferentes** usando bucles, condicionales, programación funcional y Pandas. Los datos se cruzarán para generar un **informe final en JSON** que se usará para validar el correcto funcionamiento del código.

El alumno debe organizar el proyecto siguiendo **POO**, utilizar **pruebas unitarias**, y subirlo a su **fork del repositorio base** con **GitHub Actions** para validar todo automáticamente.

Instrucciones de trabajo en el repositorio

Cuando comiences el proyecto, deberás trabajar en un **repositorio que deriva del siguiente.**

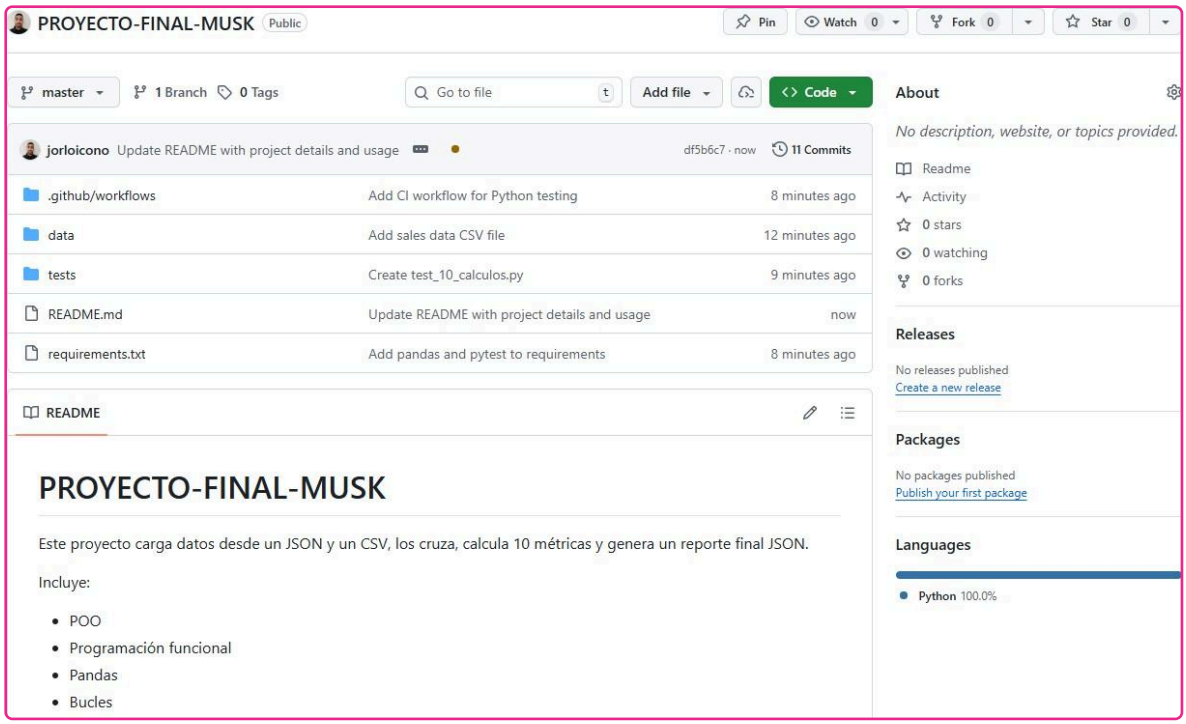
https://github.com/jorloicono/PROYECTO-FINAL-MUSK

Ese repositorio ya contiene la estructura mínima, datos, un workflow de GitHub Actions y pruebas.

Sigue cuidadosamente estas instrucciones.

Hacer Fork del Repositorio Base

1. Accede al repositorio base proporcionado por el profesor.
2. En la esquina superior derecha, haz clic en **Fork**.



3. Selecciona tu cuenta personal de GitHub como destino.
4. GitHub creará automáticamente una copia del repositorio en tu espacio.
5. Trabaja SIEMPRE en tu fork, no en el repositorio original.

A partir de ahora, cualquier cambio debes hacerlo en tu fork.

Clonar tu repositorio (fork) en tu ordenador

En tu terminal:

```
git clonehttps://github.com/TU_USUARIO/NOMBRE_DEL_REPO.git

cd NOMBRE_DEL_REPO
```

Estructura de carpetas que encontrarás en tu fork

El repositorio base tendrá una estructura similar a esta: proyecto-ventas-clientes/

```
│
├── data/
│├── clients.json
│├── sales.csv
│
│├── src/
│├── __init__.py
│├── client.py
│├── sale.py
│ ├── client_collection.py
│ ├── sales_collection.py
│ ├── functional_utils.py
│└── analyze.py ← (Aquí irá tu lógica final)
│
├── tests/
│├── test_clients.py
│├── test_sales.py
│ ├── test_calculations.py
│ ├── test_10_calculos.py
│
├── .github/
│└── workflows/
```

```
│ └── ci.yml ← GitHub Action que valida tu trabajo automáticamente
│
├── README.md
└── requirements.txt
```

¿Dónde debes subir cada archivo?

data/
Aquí NO tienes que modificar nada. Solo contiene los dos archivos iniciales:

- clients.json
- sales.csv

Estos datos se usarán para validar tus cálculos.

src/
Aquí es donde programarás TODO tu código Python.
Debes completar o editar estos archivos:

- client.py → Clase **Client**
- sale.py → Clase **Sale**
- client_collection.py → Lógica para manejar clientes.
- sales_collection.py → Lógica para manejar ventas.
- functional_utils.py → Funciones puras (filtros, mapas, transformaciones)
- analyze.py → **Script principal** que debe:
  - Leer los datos.
  - Crear los objetos.
  - Hacer los 10 cálculos requeridos.
  - Generar el informe JSON final.

El archivo **analyze.py es obligatorio completarlo.**

tests/
Aquí no debes añadir nada nuevo, pero sí debes asegurarte de que **todas las pruebas pasan.**

Los tests revisan:

- ✓ Lectura de archivos.
- ✓ Creación de objetos.
- ✓ Lógica de cálculos.
- ✓ Uso de bucles y condicionales.
- ✓ Resultado del JSON final.
- ✓ Cruce de datos entre JSON y CSV.

GitHub Actions: Comprobación automática

El repositorio incluye un **workflow de CI** en:
.github/workflows/ci.yml

Este workflow se ejecuta **AUTOMÁTICAMENTE** cuando:

- Haces un **push** a tu repositorio.
- Creas un **pull request.**
- Modificas cualquier archivo en src/ o data/

¿Qué valida GitHub Actions?

- Que el código se pueda ejecutar.
- Que todas las pruebas de la carpeta tests/ pasen.
- Que el JSON generado por analyze.py tenga:
  - Los **10 cálculos.**
  - Los nombres correctos de las claves.
  - Los valores correctos según los datos de input.

¿Cómo saber si TODO ha ido correctamente?

1. Ve a tu repositorio en GitHub.
2. En el menú superior, haz clic en **Actions.**
3. Verás un workflow llamado **CI** (o similar).
4. Si ves un ✓ verde → ¡Todo correcto!
5. Si ves un ✗ rojo → algo ha fallado. Podrás abrir el log y ver:
   - Qué cálculo está mal.
   - Por qué falló un test concreto.
   - Si hay errores de ejecución.

Señales de que lo hiciste bien:

- ✓ "All checks passed."
- ✓ Todos los tests en verde.
- ✓ El archivo JSON generado por tu código aparece en la ruta correcta
- ✓ Tu script analyze.py ejecuta los **10 cálculos** requeridos.

Clases a crear (POO)

El proyecto obliga a implementar **tres clases fundamentales** que representan la estructura del dominio: clientes, ventas y colecciones de ambos.

Clase Client
Representa un cliente individual.

Atributos
- client_id → identificador único (int)
- name → nombre del cliente (str)
- country → país del cliente (str)
- signup_date → fecha en que se registró (str o datetime)

Método requerido
- to_dict()
  Convierte el objeto a un diccionario Python para poder exportarlo como JSON sin dificultad.

Clase Sale
Representa una venta.

Atributos
- sale_id → id único de la venta (int).
- client_id → id del cliente asociado (clave externa).
- product → nombre del producto vendido.
- category → categoría (Electronics, Accessories...)
- amount → importe de la venta.
- date → fecha de la venta

Método requerido
- to_dict()
  Igual que en Client: serializa para JSON.

Clases Colección
Estas clases gestionan listas de objetos con métodos de consulta.

Clase ClientCollection

Contiene **lista de Client.**

Métodos obligatorios:

Método	Función
get_client_by_id(id)	Devuelve un Client concreto.
clients_by_country(country)	Devuelve lista de cilente de un país.

Clase salestCollection

Contiene **lista de Sale.**

Métodos obligatorios:

Método	Función
sales_by_client(client_id)	Todas las ventas de un cliente.
total_amount_by_client(client_id)	Suma de importes de un cliente.
total_amount_by_category(category)	Suma de ventas de una categoría.
average_sale_by_client(client_id)	Media de gasto por venta para un cliente.

Los 10 cálculos obligatorios

Estos cálculos deben combinar:

- Bucles.
- Condicionales.
- Programación funcional.
- Cruce/relación entre clientes y ventas.
- Pandas (para partes específicas.

1. Número total de clientes

Cuenta cuántos clientes hay en el JSON.

2. Número total de ventas

Tamaño del CSV de ventas.

3. Total de ingresos por cliente

Para cada cliente:

- Filtras sus ventas (por client_id)
- Sumas amounts.

4. Número de ventas por cliente

Simplemente la longitud de sales_by_client(id)

5. Ingreso promedio por venta de cada cliente

Promedio:
promedio = total_spent / cantidad_ventas

6. Cliente con mayor gasto por país

Cruce de datos:

1. Agrupar clientes por country (POO)
2. Calcular gastos por cliente (colección de ventas)
3. Elegir el de mayor total.

7. Total de ventas por categoría

Usando pandas:
Agrupar por categoría → sum amounts
(df.groupby("category") ["amount"].sum())

8. Cliente con más ventas en una categoría específica

Combina:

- filtro por categoría (función pura)
- filtrado por cliente.
- contar ventas.
- devolver el cliente con máximo.

9. Número de clientes que superan un gasto mínimo

Ejemplo: gasto mínimo = 500€
Requiere:

- bucles
- condicionales
- cruce de cliente + ventas (total_amount_by_client(id))

10. Ventas acumuladas mes a mes

Con pandas:

1. Convertir la fecha a datetime.
2. Extraer año-mes (df["date"]. dt.to_period("M"))
3. Agrupar (groupby) y sumar.

Programación funcional + Pandas

El proyecto pide explícitamente usar:

Programación funcional:

- funciones puras para filtrar ventas por fecha o categoría.
- uso de filter(), map(), reduce() si quieres.
- no modificar estructuras originales: siempre devolver nuevas.

Ejemplo:
def filter_sales_by_category(sales, category):
return list (filter(lambda s: s.category == category, sales))

Pandas para:

- unir datos clientes + ventas (merge)
- agrupar por categoría, país, mes.
- resumir totales.
- generar estructuras de datos listas para exportar en JSON.

Estructura final del informe JSON (explicado)

El archivo generado por analyze.py debe devolver exactamente:

```
{
"summary": { "total_clients": , "total_sales": , "total_revenue":
},
"clients": [
{
"client_id": ,
"name": , "total_spent":, "sale_count": ,

"average_sale":
},
...
],
"top_client_by_country": { "Spain": ,
"Germany": ,
"France":
},
"sales_by_category": { "Electronics": , "Accessories":
},
"high_spending_clients": [
,


],
"monthly_sales": { "2023-07":
}
}
```

Explicación campo por campo

summary

- total_clients: número total.
- total_sales: número de registros en ventas.
- total_revenue: suma absoluta de todas las ventas.

clients[]

Para cada cliente:

Campo	Explicación
total_spent	Suma de todo lo comprado
sale_count	Número de ventas
average_sale	(total_spent / sale count)

top_client_by_country

Un diccionario donde cada país tiene el cliente que más gastó.

sales_by_category

Totales agrupados por categoría.

high_spending_clients

Clientes cuyo gasto total supera un umbral (p. ej. 500€)

monthly_sales

Totales agrupados por mes (período YYYY-MM)

MUSK